

20260305_Final_Group_Assignment

Date: 2026-03-05

Paulina

```
# Load required libraries
library(readr)
library(dplyr)
library(ggplot2)
library(slides)
library(scales)

# Load dataset
Covid_cleaned <- read_csv("open_covid_data/Covid_cleaned.csv")

# Compute continent-level daily cases
continent_daily <- Covid_cleaned %>%
  mutate(
    est_new_cases = (new_cases_per_million / 1e6) * population
  ) %>%
  group_by(continent, date) %>%
  summarise(
    new_cases = sum(est_new_cases, na.rm = TRUE),
    pop = sum(population, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    new_cases_per_million_cont = (new_cases / pop) * 1e6
```

```

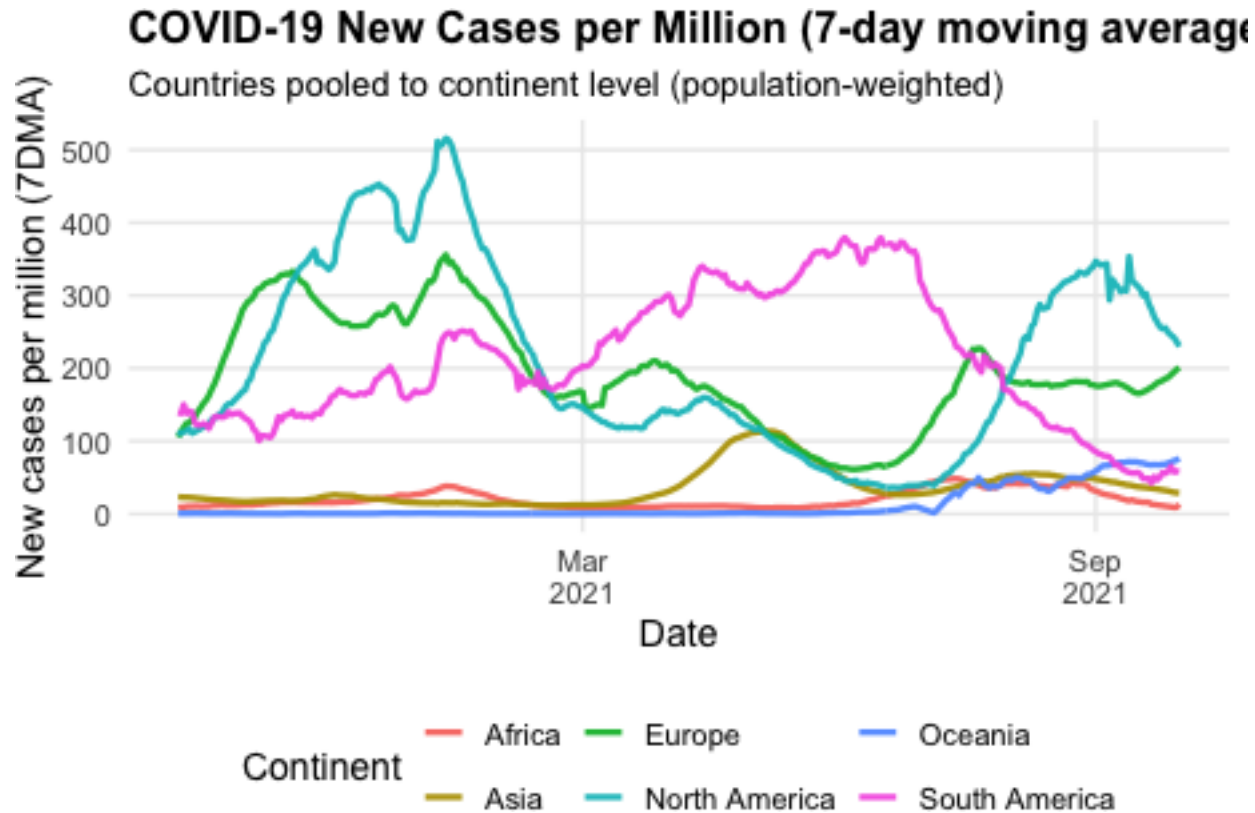
) %>%
group_by(continent) %>%
arrange(continent, date) %>%
mutate(
  new_cases_pm_7dma = slider::slide_dbl(
    new_cases_per_million_cont,
    mean,
    .before = 6,
    .complete = TRUE,
    na.rm = TRUE
  )
) %>%
ungroup()

# Remove rows with NA moving averages
continent_daily <- continent_daily %>%
  filter(!is.na(new_cases_pm_7dma))

# Plot results
ggplot(continent_daily, aes(x = date, y = new_cases_pm_7dma, color = continent)) +
  geom_line(linewidth = 1.1, alpha = 0.95) +
  scale_x_date(date_breaks = "6 months", date_labels = "%b\n%Y") +
  scale_y_continuous(labels = label_number(accuracy = 1)) +
  labs(
    title = "COVID-19 New Cases per Million (7-day moving average)",
    subtitle = "Countries pooled to continent level (population-weighted)",
    x = "Date",
    y = "New cases per million (7DMA)",
    color = "Continent"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    plot.subtitle = element_text(size = 13),
    legend.position = "bottom",

```

```
panel.grid.minor = element_blank()
)
```



Lin

This analysis uses country-level COVID data from all continents. Countries are classified into four UNDP HDI categories (Low, Medium, High, Very high HDI). Daily new cases per million are aggregated to monthly means within each country, then averaged across countries (unweighted) within each HDI category. The plot shows monthly means with 95% CI.

```

# Load required libraries
library(readr)
library(dplyr)
library(lubridate)
library(ggplot2)

# 1) Read cleaned COVID dataset
PATH_COVID <- file.path("open_covid_data", "Covid_cleaned.csv")
covid_raw <- readr::read_csv(PATH_COVID, show_col_types = FALSE)

# 2) Rename variables and set types
recodedCovid <- covid_raw %>%
  dplyr::rename(
    country = location,
    num_new_cases_per_million = new_cases_per_million,
    num_hdi = human_development_index
  ) %>%
  dplyr::mutate(
    date = as.Date(date),
    continent = as.character(continent),
    country = as.character(country),
    num_new_cases_per_million = as.numeric(num_new_cases_per_million),
    num_hdi = as.numeric(num_hdi)
  )

# 3) Keep country rows (all continents) and required values
recodedCovid <- recodedCovid %>%
  dplyr::filter(
    !is.na(continent),
    !is.na(date),
    !is.na(country),
    !is.na(num_new_cases_per_million),
    !is.na(num_hdi)
  )

```

```

# 4) Create monthly time variable
recodedCovid <- recodedCovid %>%
  dplyr::mutate(month = lubridate::floor_date(date, "month"))

# 5) Compute one HDI value per country (mean across available rows) + HDI groups
hdiCountry <- recodedCovid %>%
  dplyr::group_by(country) %>%
  dplyr::summarise(
    num_hdi_country = mean(num_hdi, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    hdi_group = dplyr::case_when(
      num_hdi_country < 0.550 ~ "Low HDI",
      num_hdi_country < 0.700 ~ "Medium HDI",
      num_hdi_country < 0.800 ~ "High HDI",
      num_hdi_country >= 0.800 ~ "Very high HDI",
      TRUE ~ NA_character_
    ),
    hdi_group = factor(
      hdi_group,
      levels = c("Low HDI", "Medium HDI", "High HDI", "Very high HDI")
    )
  )

# Join HDI group back to daily data
recodedCovid <- recodedCovid %>%
  dplyr::left_join(
    hdiCountry %>% dplyr::select(country, hdi_group),
    by = "country"
  ) %>%
  dplyr::filter(!is.na(hdi_group))

# 6) Within each country, compute monthly mean of daily new cases per million
countryMonth <- recodedCovid %>%

```

```

dplyr::group_by(country, hdi_group, month) %>%
dplyr::summarise(
  monthly_mean_new_cases_pm = mean(num_new_cases_per_million, na.rm = TRUE),
  .groups = "drop"
)

# 7) Within each HDI group and month, compute unweighted country average + 95% CI
groupMonth <- countryMonth %>%
dplyr::group_by(hdi_group, month) %>%
dplyr::summarise(
  mean_value = mean(monthly_mean_new_cases_pm, na.rm = TRUE),
  sd_value = sd(monthly_mean_new_cases_pm, na.rm = TRUE),
  n_countries = dplyr::n(),
  se = sd_value / sqrt(n_countries),
  lower_95 = mean_value - 1.96 * se,
  upper_95 = mean_value + 1.96 * se,
  .groups = "drop"
) %>%
dplyr::arrange(hdi_group, month)

# 8) Plot global monthly trend by HDI category
p_final <- ggplot(groupMonth, aes(x = month, y = mean_value, colour = hdi_group, fill = hdi_group)) +
  geom_ribbon(aes(ymin = lower_95, ymax = upper_95),
    alpha = 0.18, colour = NA, na.rm = TRUE) +
  geom_line(linewidth = 1.1, na.rm = TRUE) +
  scale_x_date(
    date_breaks = "1 months",
    labels = function(x) paste0(month.abb[lubridate::month(x)], ". ", lubridate::year(x))
  ) +
  labs(
    x = "Month",
    y = "mean new cases per million",
    title = "Global Trends in monthly New COVID-19 Cases per Million by HDI category",
    colour = "HDI category",
    fill = "HDI category"
  )

```

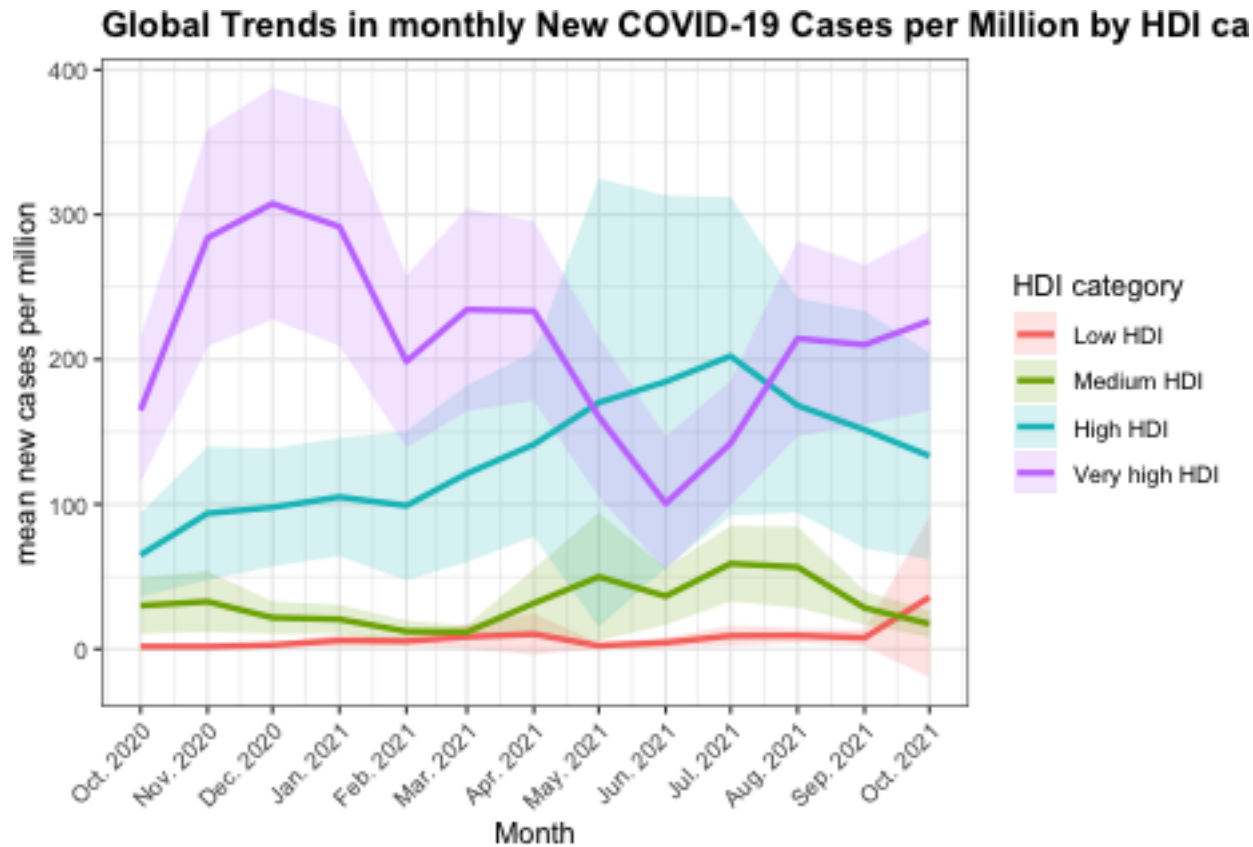
```

) +
theme_bw() +
theme(
  axis.text.x = element_text(angle = 45, hjust = 1),
  plot.title = element_text(face = "bold")
)

# Save figure
dir.create(file.path("outputs", "figures"), recursive = TRUE, showWarnings = FALSE)
ggsave(
  filename = file.path("outputs", "figures", "global_hdi_newcases_monthlymean_unweighted.png"),
  plot = p_final,
  width = 10, height = 6, dpi = 300
)

# Print plot
p_final

```



This figure shows global monthly trends in new COVID-19 cases per million across countries grouped by the Human Development Index (HDI). HDI is a composite indicator used to reflect a country's overall level of development, typically incorporating dimensions such as health, education, and income.

Methodologically, daily new cases per million were aggregated to monthly means within each country, and then an unweighted average across countries was calculated within each HDI category for each month. An unweighted average was used so that each country contributed equally, allowing the figure to better reflect country-level average patterns within HDI categories rather than being dominated by a small number of highly populated countries. HDI groups were defined using the UNDP standard classification and the Shaded bands indicate 95% CIs around the monthly group mean.

The figure shows clear differences in monthly new COVID-19 case trends across HDI categories, but these differences are not constant over time. Overall, the high and very high HDI groups reported higher case levels than the medium and low HDI groups in most months, suggesting generally higher reported new-case rates in countries with higher development levels. However, the size of these differences changed over time, as the groups did not follow the same timing or pattern of increase, decline, and rebound. This indicates that HDI-related differences were evident not only in magnitude, but also in the shape and timing of trends. The 95% CIs reflect uncertainty around the group means and within-group variability; when the shaded bands are wide or substantially overlap, between-group differences should be interpreted cautiously.

This figure highlights heterogeneity in global COVID-19 trends across HDI categories from a development-stratified perspective, and provides information for discussing possible differences in surveillance and reporting systems, epidemic timing, and public health responses. However, this is a descriptive visualization and should not be interpreted causally.

Shiqiu (Q)

```
# =====
# Cumulative COVID-19 Deaths per Million - Interactive Maps
#   - Flat world + Rotatable globe
#   - Uses total_deaths_per_million at each country's latest date
#
# Path principles (no hard-coded script folder name):
#   - Input read from PROJECT ROOT via here::here(...)
#   - Output written next to this script via knitr::current_input()
# =====

# =====
# 0) Packages (install if missing; load quietly)
# =====
required_packages <- c(
  "readr", "dplyr", "here", "countrycode",
  "plotly", "rnatrualearth", "sf",
  "htmlwidgets", "knitr", "rstudioapi",
  "webshot2"
)
```

```

for (pkg in required_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg, dependencies = TRUE)
  }
  suppressPackageStartupMessages(
    library(pkg, character.only = TRUE)
  )
}

# =====
# 1) Output path helpers (no hard-coding)
# - out_dir = folder containing this script
# - out_path("x.csv") returns full path inside out_dir
# =====
get_this_script_dir <- function() {
  # --- 1) Knitting context (Rmd / Quarto)
  p <- knitr::current_input()
  if (!is.null(p) && nzchar(p)) {
    return(normalizePath(dirname(p), winslash = "/", mustWork = FALSE))
  }

  # --- 2) Interactive RStudio context
  if (rstudioapi::isAvailable()) {
    ctx <- rstudioapi::getActiveDocumentContext()
    if (!is.null(ctx$path) && nzchar(ctx$path)) {
      return(normalizePath(dirname(ctx$path), winslash = "/", mustWork = FALSE))
    }
  }

  # --- 3) Fallback
  normalizePath(getwd(), winslash = "/", mustWork = FALSE)
}

out_dir <- get_this_script_dir()

```

```

out_path <- function(...) {
  file.path(out_dir, ...)
}

if (!dir.exists(out_dir)) dir.create(out_dir, recursive = TRUE)

# =====
# 2) Small helpers (robustness)
# =====
first_non_na <- function(x) {
  x2 <- x[!is.na(x)]
  if (length(x2) == 0) NA else x2[[1]]
}

# =====
# 3) Read dataset (PROJECT ROOT)
# =====
csv_path <- here::here("open_covid_data", "Covid_cleaned.csv")
if (!file.exists(csv_path)) stop("File not found: ", csv_path)

df <- readr::read_csv(csv_path, show_col_types = FALSE)

needed_cols <- c("location", "continent", "date", "total_deaths_per_million")
missing_cols <- setdiff(needed_cols, names(df))
if (length(missing_cols) > 0) {
  stop("Missing required columns in CSV: ", paste(missing_cols, collapse = ", "))
}

df <- df %>%
  dplyr::mutate(date = as.Date(date))

# =====

```

```

# 4) Name -> ISO3 conversion (with minimal targeted fixes)
# =====
name_fixes <- c(
  "Timor" = "Timor-Leste",
  "East Timor" = "Timor-Leste",
  "Congo" = "Republic of the Congo",
  "Congo (Brazzaville)" = "Republic of the Congo",
  "Congo (Kinshasa)" = "Democratic Republic of the Congo",
  "DR Congo" = "Democratic Republic of the Congo",
  "Czechia" = "Czech Republic",
  "Cape Verde" = "Cabo Verde",
  "Swaziland" = "Eswatini",
  "Ivory Coast" = "Côte d'Ivoire",
  "Cote d'Ivoire" = "Côte d'Ivoire",
  "Cote dIvoire" = "Côte d'Ivoire",
  "Myanmar (Burma)" = "Myanmar",
  "North Macedonia" = "Macedonia"
)

df <- df %>%
  dplyr::mutate(
    location_clean = dplyr::if_else(
      location %in% names(name_fixes),
      unname(name_fixes[location]),
      location
    ),
    iso_code = countrycode::countrycode(
      sourcevar = location_clean,
      origin = "country.name",
      destination = "iso3c",
      warn = TRUE
    )
  )

stopifnot("iso_code" %in% names(df))

```

```

# =====
# 5) Location-level diagnostics (why rows become unmapped)
# =====
loc_diag <- df %>%
  dplyr::group_by(location, location_clean) %>%
  dplyr::summarise(
    continent_any = any(!is.na(continent)),
    iso_code_any = first_non_na(iso_code),
    has_iso = any(!is.na(iso_code)),
    has_deaths_pm = any(!is.na(total_deaths_per_million)),
    deaths_pm_any = first_non_na(total_deaths_per_million),
    latest_date_any = suppressWarnings(max(date, na.rm = TRUE)),
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    reason = dplyr::case_when(
      !continent_any ~ "Excluded by filter: continent is NA (aggregate/non-country row)",
      !has_iso ~ "ISO conversion failed (countrycode could not map name to ISO3)",
      has_iso & !has_deaths_pm ~ "total_deaths_per_million missing in dataset for this country",
      TRUE ~ "OK"
    )
  )

loc_diag_iso <- loc_diag %>%
  dplyr::filter(!is.na(iso_code_any)) %>%
  dplyr::group_by(iso_code_any) %>%
  dplyr::summarise(
    any_deaths_missing = any(reason == "total_deaths_per_million missing in dataset for this country", na.rm = TRUE),
    any_excluded_by_continent = any(reason == "Excluded by filter: continent is NA (aggregate/non-country row)", na.rm = TRUE),
    any_iso_failed = any(reason == "ISO conversion failed (countrycode could not map name to ISO3)", na.rm = TRUE),
    .groups = "drop"
  )

```

```

# =====
# 6) World polygons index (Natural Earth)
#   - Uses iso_a3 with adm0_a3 fallback
# =====
world_sf <- rnaturalearth::ne_countries(scale = "medium", returnclass = "sf")

world_index <- world_sf %>%
  sf::st_drop_geometry() %>%
  dplyr::transmute(
    iso_a3 = dplyr::if_else(is.na(iso_a3) | iso_a3 == "-99", adm0_a3, iso_a3),
    name
  ) %>%
  dplyr::distinct() %>%
  dplyr::filter(!is.na(iso_a3), iso_a3 != "-99")

# =====
# 7) Country-level latest cumulative deaths per million
#   - Filters to country rows (continent != NA)
#   - Selects latest date per ISO3 country
# =====
deaths_country <- df %>%
  dplyr::filter(!is.na(continent)) %>%
  dplyr::filter(!is.na(iso_code)) %>%
  dplyr::group_by(iso_code) %>%
  dplyr::filter(date == suppressWarnings(max(date, na.rm = TRUE))) %>%
  dplyr::summarise(
    country      = first_non_na(location_clean),
    latest_date   = suppressWarnings(max(date, na.rm = TRUE)),
    deaths_pm_latest = first_non_na(total_deaths_per_million),
    .groups = "drop"
  ) %>%
  dplyr::filter(!is.na(deaths_pm_latest)) %>%
  dplyr::mutate(
    hover = paste0(

```

```

    "<b>", country, "</b>",
    "<br>ISO3: ", iso_code,
    "<br>Latest date: ", as.character/latest_date),
    "<br>Cumulative deaths per million: ",
    format(round(deaths_pm_latest, 1), nsmall = 1, big.mark = ",")
  )
)

latest_date_overall <- suppressWarnings(max(df$date, na.rm = TRUE))

# =====
# 8) Polygons missing from mapped data (diagnostic join)
# =====
missing_on_map <- world_index %>%
  dplyr::left_join(
    deaths_country %>% dplyr::transmute(iso_a3 = iso_code, has_deaths = TRUE),
    by = "iso_a3"
  ) %>%
  dplyr::filter(is.na(has_deaths)) %>%
  dplyr::select(iso_a3, name) %>%
  dplyr::left_join(loc_diag_iso, by = c("iso_a3" = "iso_code_any")) %>%
  dplyr::mutate(
    reason = dplyr::case_when(
      !is.na(any_deaths_missing) & any_deaths_missing ~ "total_deaths_per_million missing in dataset for this count",
      !is.na(any_excluded_by_continent) & any_excluded_by_continent ~ "Present in dataset but excluded by continent==NA filter",
      !is.na(any_iso_failed) & any_iso_failed ~ "ISO conversion failed in dataset (name mismatch / non-standard)",
      TRUE ~ "No matching deaths-per-million record in dataset"
    ),
    hover = paste0(
      "<b>", name, "</b>",
      "<br>ISO3: ", iso_a3,
      "<br>Status: Missing from deaths-per-million map",
      "<br>Reason: ", reason
    )
  )
)

```

```

) %>%
dplyr::select(iso_a3, name, reason, hover)

# =====
# 9) Diagnostics exports (next to this script)
# =====

readr::write_csv(
  deaths_country %>%
    dplyr::select(country, iso_code, latest_date, deaths_pm_latest) %>%
    dplyr::arrange(country),
  out_path("non_missing_cum_deaths_per_million_countries.csv")
)

readr::write_csv(
  missing_on_map %>%
    dplyr::rename(iso_code = iso_a3, country = name) %>%
    dplyr::arrange(country),
  out_path("missing_cum_deaths_per_million_or_join_countries.csv")
)

iso_failed_locations <- loc_diag %>%
  dplyr::filter(reason == "ISO conversion failed (countrycode could not map name to ISO3)") %>%
  dplyr::select(location, location_clean, reason) %>%
  dplyr::arrange(location_clean)

readr::write_csv(
  iso_failed_locations,
  out_path("iso_conversion_failed_locations.csv")
)

all_world_polygons_with_status <- world_index %>%
  dplyr::left_join(
    deaths_country %>% dplyr::select(iso_code, country, latest_date, deaths_pm_latest),
    by = c("iso_a3" = "iso_code")
  )

```

```

) %>%
dplyr::left_join(loc_diag_iso, by = c("iso_a3" = "iso_code_any")) %>%
dplyr::mutate(
  status = dplyr::if_else(!is.na(deaths_pm_latest), "Non-missing (mapped)", "Missing (grey)"),
  reason = dplyr::case_when(
    status == "Non-missing (mapped)" ~ "OK",
    !is.na(any_deaths_missing) & any_deaths_missing ~ "total_deaths_per_million missing in dataset for this c",
    !is.na(any_excluded_by_continent) & any_excluded_by_continent ~ "Present in dataset but excluded by continent==NA filter",
    !is.na(any_iso_failed) & any_iso_failed ~ "ISO conversion failed in dataset (name mismatch / non-",
    TRUE ~ "Not present in dataset (or not convertible via dataset loc
  )
) %>%
dplyr::transmute(
  iso_code = iso_a3,
  polygon_name = name,
  status,
  reason,
  latest_date,
  deaths_pm_latest
) %>%
dplyr::arrange(status, polygon_name)

readr::write_csv(
  all_world_polygons_with_status,
  out_path("all_world_polygons_with_status.csv")
)

cat("Diagnostics exported to:\n")

```

Diagnostics exported to:

```
cat(" - ", out_path("non_missing_cum_deaths_per_million_countries.csv"), "\n", sep = "")
```

- /Users/qiuqiumeng/Projects/2026-Group2-SQMeng/non_missing_cum_deaths_per_million_countries.csv

```
cat(" - ", out_path("missing_cum_deaths_per_million_or_join_countries.csv"), "\n", sep = "")
```

- /Users/qiuqiumeng/Projects/2026-Group2-SQMeng/missing_cum_deaths_per_million_or_join_countries.csv

```
cat(" - ", out_path("iso_conversion_failed_locations.csv"), "\n", sep = "")
```

- /Users/qiuqiumeng/Projects/2026-Group2-SQMeng/iso_conversion_failed_locations.csv

```
cat(" - ", out_path("all_world_polygons_with_status.csv"), "\n\n", sep = "")
```

- /Users/qiuqiumeng/Projects/2026-Group2-SQMeng/all_world_polygons_with_status.csv

```
cat("Latest date overall in dataset: ", as.character(latest_date_overall), "\n", sep = "")
```

Latest date overall in dataset: 2021-10-01

```
cat("ISO conversion failures (rows): ", sum(is.na(df$iso_code)), "\n", sep = "")
```

ISO conversion failures (rows): 0

```
cat("Unique locations failing ISO conversion: ", nrow(iso_failed_locations), "\n\n", sep = "")
```

Unique locations failing ISO conversion: 0

```
# =====  
# 10) Map data prep (NA retained for missing; no 0 imputation)  
# =====  
map_data <- world_index %>%  
  dplyr::left_join(  
    deaths_country %>% dplyr::select(iso_code, deaths_pm_latest, hover),  
    by = c("iso_a3" = "iso_code")  
  ) %>%  
  dplyr::mutate(  
    hover = dplyr::if_else(  
      is.na(deaths_pm_latest),  
      paste0(  
        "<b>", name, "</b>",  
        "<br>ISO3: ", iso_a3,  
        "<br>Status: Missing cumulative deaths per million"  
      )  
    )  
  )
```

```

    ),
    hover
  )
)

map_non_missing <- map_data %>% dplyr::filter(!is.na(deaths_pm_latest))
map_missing     <- map_data %>% dplyr::filter(is.na(deaths_pm_latest))

# =====
# 11) Palette + robust zmax (99th percentile)
# =====
colorscale_deaths <- list(
  list(0.00, "#FFFFCC"),
  list(0.20, "#FFEDA0"),
  list(0.35, "#FED976"),
  list(0.50, "#FEB24C"),
  list(0.65, "#FD8D3C"),
  list(0.80, "#FC4E2A"),
  list(0.92, "#E31A1C"),
  list(1.00, "#800026")
)

zmax_deaths <- stats::quantile(map_non_missing$deaths_pm_latest, probs = 0.99, na.rm = TRUE)
if (!is.finite(zmax_deaths) || zmax_deaths <= 0) {
  zmax_deaths <- max(map_non_missing$deaths_pm_latest, na.rm = TRUE)
}

# =====
# 12) Flat world map (2 traces: coloured + grey overlay)
# =====
base_geo_flat <- list(
  showframe      = FALSE,
  showcoastlines = FALSE,

```

```

projection      = list(type = "natural earth"),
showocean       = TRUE,
oceancolor      = "#FAFAFA",
showland        = TRUE,
landcolor       = "#FFFFFF",
showcountries   = TRUE,
countrycolor    = "#FFFFFF"
)

map_flat <- plotly::plot_ly() %>%
  plotly::add_trace(
    data          = map_non_missing,
    type          = "choropleth",
    locations     = ~iso_a3,
    locationmode  = "ISO-3",
    z             = ~deaths_pm_latest,
    text         = ~hover,
    hoverinfo     = "text",
    colorscale    = colorscale_deaths,
    autocolorscale = FALSE,
    zauto         = FALSE,
    zmin         = 0,
    zmax         = zmax_deaths,
    showscale     = TRUE,
    colorbar      = list(title = "Cumulative deaths / 1M", tickformat = ",.0f"),
    marker        = list(line = list(color = "#FFFFFF", width = 0.3))
  ) %>%
  plotly::add_trace(
    data          = map_missing,
    type          = "choropleth",
    locations     = ~iso_a3,
    locationmode  = "ISO-3",
    z             = I(1),
    text         = ~hover,
    hoverinfo     = "text",

```

```

    colorscale = list(list(0, "#BDBDBD"), list(1, "#BDBDBD")),
    showscale = FALSE,
    marker     = list(line = list(color = "#FFFFFF", width = 0.3))
  ) %>%
  plotly::layout(
    title = paste0(
      "Cumulative COVID-19 deaths per million",
      "<br>(flat world; overall latest = ",
      as.character(latest_date_overall),
      ")"
    ),
    geo = base_geo_flat,
    margin = list(l = 0, r = 0, t = 70, b = 0)
  )

if (knitr::is_latex_output()) {
  if (!requireNamespace("webshot2", quietly = TRUE)) {
    stop("Package 'webshot2' is required to include Plotly widgets in PDF. Install with install.packages('webshot2').")
  }

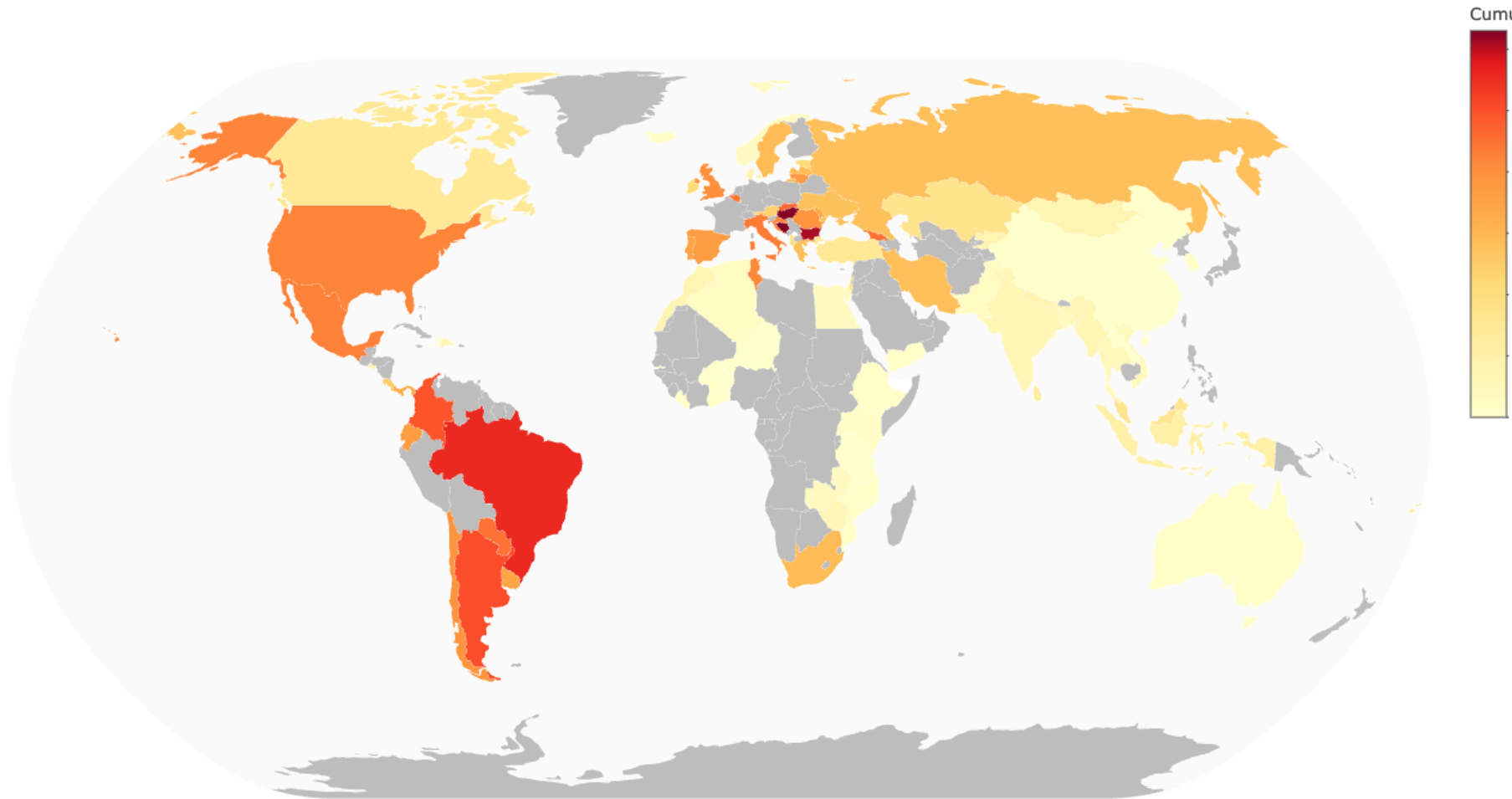
  flat_tmp_html <- out_path("tmp_flat_map.html")
  flat_png <- out_path("CumDeathsPerMillion_FlatWorld.png")

  htmlwidgets::saveWidget(map_flat, flat_tmp_html, selfcontained = TRUE)
  webshot2::webshot(flat_tmp_html, flat_png, vwidth = 1400, vheight = 800)

  knitr::include_graphics(flat_png)
} else {
  map_flat
}

```

Cumulative COVID-19 deaths per million
(flat world; overall latest = 2021-10-01)



```

map_flat$-sizingPolicy <- htmlwidgets::sizingPolicy(
  browser.fill = TRUE,
  viewer.fill  = TRUE,
  padding      = 0
)

flat_html  <- out_path("CumDeathsPerMillion_FlatWorld.html")
flat_libdir <- out_path("CumDeathsPerMillion_FlatWorld_files")

htmlwidgets::saveWidget(
  widget      = map_flat,
  file        = flat_html,
  selfcontained = FALSE,
  libdir      = flat_libdir
)

# browseURL(flat_html)

# =====
# 13) Rotatable globe (orthographic; same 2-trace logic)
# =====

geo_globe <- list(
  projection = list(type = "orthographic", rotation = list(lon = 10, lat = 20)),
  showframe  = FALSE,
  showocean  = TRUE,
  oceancolor = "#FAFAFA",
  showland   = TRUE,
  landcolor  = "#FFFFFF",
  showcoastlines = TRUE,
  coastlinecolor = "rgba(0,0,0,0.15)",
  showcountries = TRUE,
  countrycolor = "#FFFFFF",
  showlakes    = TRUE,
  lakecolor    = "#EBF5FF"
)

```

```

)

map_globe <- plotly::plot_ly() %>%
  plotly::add_trace(
    data      = map_non_missing,
    type      = "choropleth",
    locations = ~iso_a3,
    locationmode = "ISO-3",
    z         = ~deaths_pm_latest,
    text      = ~hover,
    hoverinfo = "text",
    colorscale = colorscale_deaths,
    autocolorscale = FALSE,
    zauto      = FALSE,
    zmin       = 0,
    zmax       = zmax_deaths,
    showscale  = TRUE,
    colorbar   = list(title = "Cumulative deaths / 1M", tickformat = ",.0f"),
    marker     = list(line = list(color = "#FFFFFF", width = 0.4))
  ) %>%
  plotly::add_trace(
    data      = map_missing,
    type      = "choropleth",
    locations = ~iso_a3,
    locationmode = "ISO-3",
    z         = I(1),
    text      = ~hover,
    hoverinfo = "text",
    colorscale = list(list(0, "#BDBDBD"), list(1, "#BDBDBD")),
    showscale  = FALSE,
    marker     = list(line = list(color = "#FFFFFF", width = 0.4))
  ) %>%
  plotly::layout(
    title = paste0(
      "Cumulative COVID-19 deaths per million",

```

```

    "<br>(rotatable globe; overall latest = ",
    as.character(latest_date_overall),
    ")"
  ),
  geo      = geo_globe,
  margin = list(l = 0, r = 0, t = 70, b = 0)
) %>%
plotly::config(displayModeBar = TRUE, scrollZoom = TRUE)

if (knitr::is_latex_output()) {
  if (!requireNamespace("webshot2", quietly = TRUE)) {
    stop("Package 'webshot2' is required to include Plotly widgets in PDF. Install with install.packages('webshot2').")
  }

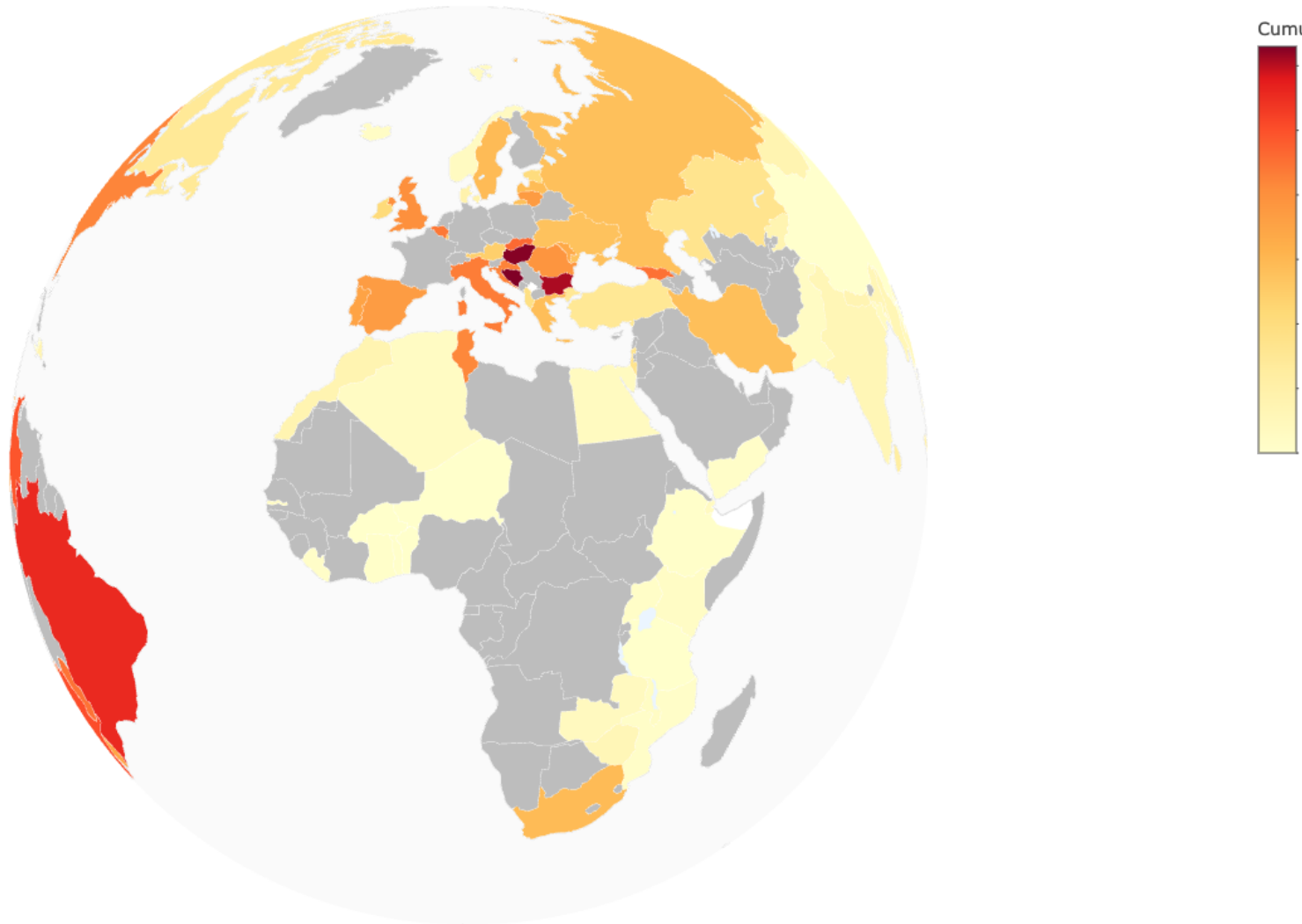
  globe_tmp_html <- out_path("tmp_globe_map.html")
  globe_png <- out_path("CumDeathsPerMillion_RotatableGlobe.png")

  htmlwidgets::saveWidget(map_globe, globe_tmp_html, selfcontained = TRUE)
  webshot2::webshot(globe_tmp_html, globe_png, vwidth = 1400, vheight = 800)

  knitr::include_graphics(globe_png)
} else {
  map_globe
}

```

Cumulative COVID-19 deaths per million
(rotatable globe; overall latest = 2021-10-01)



```

map_globe$ sizingPolicy <- htmlwidgets::sizingPolicy(
  browser.fill = TRUE,
  viewer.fill  = TRUE,
  padding      = 0
)

globe_html <- out_path("CumDeathsPerMillion_RotatableGlobe.html")
globe_libdir <- out_path("CumDeathsPerMillion_RotatableGlobe_files")

htmlwidgets::saveWidget(
  widget      = map_globe,
  file        = globe_html,
  selfcontained = FALSE,
  libdir      = globe_libdir
)

# browseURL(globe_html)

```